

Università degli Studi di Modena e Reggio Emilia

Corso di Laurea in Informatica

Assignment Two

Dataflow Analysis

Compilatori — Middle End

Prof. Andrea Marongiu

Anno accademico 2025/2026

Indice

1	Introduzione	2
2	Very Busy Expressions	2
2.1	Struttura del CFG	3
2.2	Framework di Dataflow Analysis	3
2.2.1	Gen e Kill per blocco	3
2.3	Algoritmo iterativo	3
2.4	Tabella delle iterazioni	4
2.4.1	Iterazione 0 (inizializzazione)	4
2.4.2	Iterazione 1	4
2.4.3	Iterazione 2	4
2.5	Risultato finale	4
3	Dominator Analysis	5
3.1	Struttura del CFG	5
3.2	Framework di Dataflow Analysis	6
3.2.1	Gen e Kill	6
3.3	Algoritmo iterativo	6
3.4	Tabella delle iterazioni	6
3.4.1	Iterazione 0 (inizializzazione)	6
3.4.2	Iterazione 1	6
3.4.3	Iterazione 2	7
3.4.4	Iterazione 3	7
3.4.5	Iterazione 4	7
3.5	Risultato finale	7
4	Constant Propagation	8
4.1	Struttura del CFG	9
4.2	Framework di Dataflow Analysis	9
4.2.1	Regole simboliche per $\cap$ e $\setminus$	9
4.2.2	Transfer function dinamica	9
4.3	Algoritmo iterativo	10
4.4	Tabella delle iterazioni	10
4.4.1	Iterazione 0 (inizializzazione)	10
4.4.2	Iterazione 1	10
4.4.3	Iterazione 2	10
4.4.4	Iterazione 3	11
4.5	Risultato finale	11

1 Introduzione

Per ciascuna delle tre analisi di dataflow richieste dal PDF dell'assignment:

1. deriviamo la **formalizzazione** del framework (dominio, direzione, transfer function, meet, boundary, initial points);
2. popoliamo la **tabella iterativa** sul CFG di esempio fino al fixed point.

2 Very Busy Expressions

Analisi **backward** sul CFG di esempio. Un'espressione è *very busy* in un punto p se, su **tutti** i percorsi da p a EXIT, viene valutata **prima** che uno dei suoi operandi venga ridefinito.

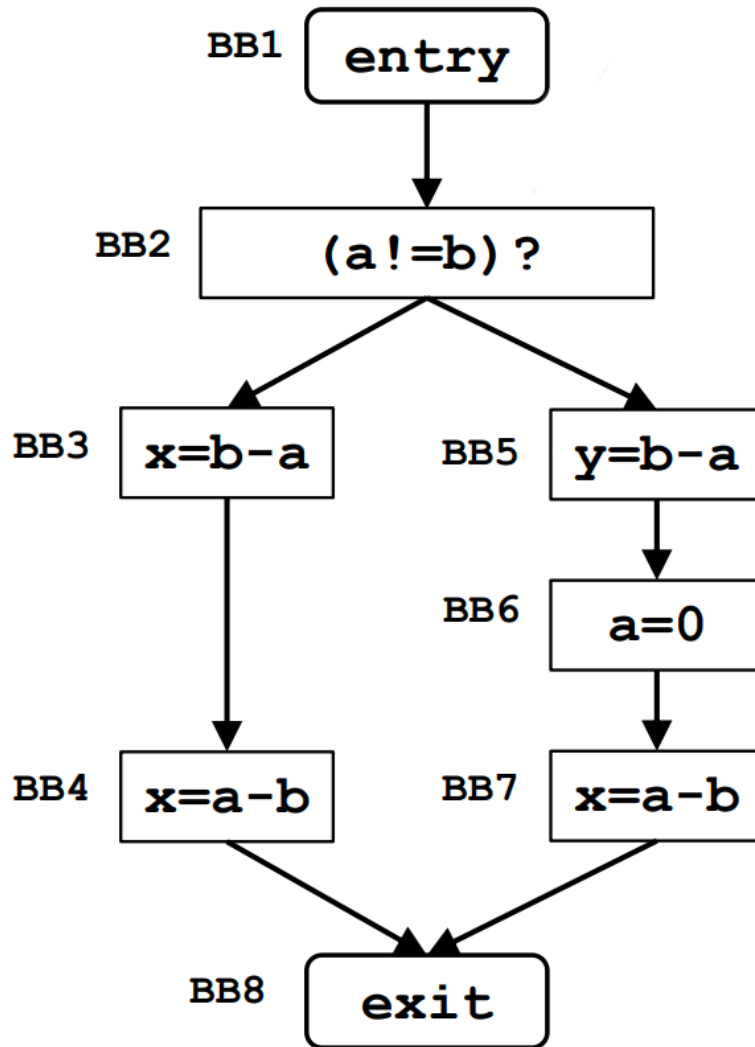


Figura 1: CFG Very Busy Expressions

2.1 Struttura del CFG

Blocco	Contenuto	Successori
BB1 (entry)	—	BB2
BB2	(a != b) ?	BB3, BB5
BB3	x = b - a	BB4
BB4	x = a - b	BB8
BB5	y = b - a	BB6
BB6	a = 0	BB7
BB7	x = a - b	BB8
BB8 (exit)	—	—

Legenda: $e_1 = b - a$, $e_2 = a - b$.

Dominio (solo espressioni **aritmetiche** nel CFG):

- $\mathcal{E} = \{e_1, e_2\} = \{b - a, a - b\}$
- $\mathcal{U} = \{e_1, e_2\}$

La condizione (a != b) in BB2 è un **branch**, non un'espressione del dominio. L'analisi Very Busy riguarda le espressioni aritmetiche da eventualmente sollevare (code hoisting).

2.2 Framework di Dataflow Analysis

Parametro	Valore
Dominio	$\mathcal{P}(\mathcal{E})$, $\mathcal{E} = \{e_1, e_2\}$
Direzione	Backward
Equazioni	$\text{IN}(B) = f_B(\text{OUT}(B))$; $\text{OUT}(B) = \bigwedge_{S \in \text{succ}(B)} \text{IN}(S)$
Transfer function	$f_B(x) = \text{Gen}_B \cup (x \setminus \text{Kill}_B)$
Meet operator ($\bigwedge$)	Intersezione $\cap$
Boundary condition	$\text{IN}(\text{EXIT}) = \text{IN}(\text{BB8}) = \emptyset$
Initial interior points	$\text{IN}(B) = \mathcal{U}$ per ogni $B \neq \text{BB8}$

Tabella 1: Formalizzazione Very Busy Expressions

2.2.1 Gen e Kill per blocco

Blocco	Gen _B	Kill _B	Note
BB1	$\emptyset$	$\emptyset$	entry vuota
BB2	$\emptyset$	$\emptyset$	solo branch (nessuna expr. aritm.)
BB3	$\{e_1\}$	$\emptyset$	b-a valutata prima di x
BB4	$\{e_2\}$	$\emptyset$	a-b valutata prima di x
BB5	$\{e_1\}$	$\emptyset$	b-a valutata prima di y
BB6	$\emptyset$	$\{e_1, e_2\}$	a = 0 invalida entrambe
BB7	$\{e_2\}$	$\emptyset$	a-b valutata
BB8	$\emptyset$	$\emptyset$	exit

2.3 Algoritmo iterativo

Per ogni iterazione, si visitano i blocchi **dal exit verso l'entry**:

1. $\text{OUT}(B) = \bigcap_{S \in \text{succ}(B)} \text{IN}(S)$ (se nessun successore: $\emptyset$)
2. $\text{IN}(B) = \text{Gen}_B \cup (\text{OUT}(B) \setminus \text{Kill}_B)$

Inizializzazione: $\text{IN}(B) = \mathcal{U}$ per BB1–BB7; $\text{IN}(\text{BB8}) = \emptyset$.

2.4 Tabella delle iterazioni

2.4.1 Iterazione 0 (inizializzazione)

Blocco	IN (inizio)
BB1 ... BB7	$\{e_1, e_2\}$
BB8	$\emptyset$

2.4.2 Iterazione 1

Blocco	IN (inizio)	OUT (calcolato)	IN (fine)	Ch.?
BB8	$\emptyset$	$\emptyset$	$\emptyset$	No
BB7	$\{e_1, e_2\}$	$\text{IN}(\text{BB8}) = \emptyset$	$\{e_2\}$	Sì
BB6	$\{e_1, e_2\}$	$\text{IN}(\text{BB7}) = \{e_2\}$	$\emptyset$	Sì
BB5	$\{e_1, e_2\}$	$\text{IN}(\text{BB6}) = \emptyset$	$\{e_1\}$	Sì
BB4	$\{e_1, e_2\}$	$\text{IN}(\text{BB8}) = \emptyset$	$\{e_2\}$	Sì
BB3	$\{e_1, e_2\}$	$\text{IN}(\text{BB4}) = \{e_2\}$	$\{e_1, e_2\}$	Sì
BB2	$\{e_1, e_2\}$	$\text{IN}(\text{BB3}) \cap \text{IN}(\text{BB5}) = \{e_1\}$	$\{e_1\}$	Sì
BB1	$\{e_1, e_2\}$	$\text{IN}(\text{BB2}) = \{e_1\}$	$\{e_1\}$	Sì

Dettaglio BB3: $\text{IN} = \{e_1\} \cup (\{e_2\} \setminus \emptyset) = \{e_1, e_2\}$

Dettaglio BB2: $\text{OUT} = \{e_1, e_2\} \cap \{e_1\} = \{e_1\}$ (solo e_1 è busy su **entrambi** i rami)

2.4.3 Iterazione 2

Nessun blocco cambia — **fixed point raggiunto**.

2.5 Risultato finale

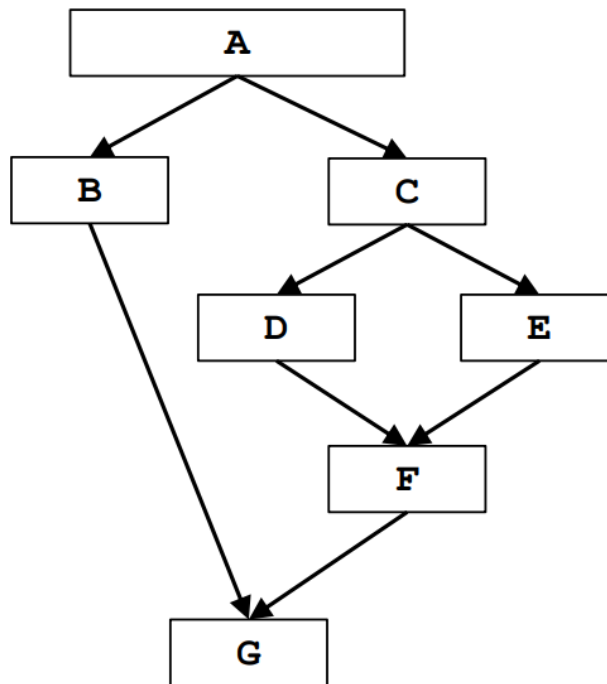
Blocco	IN	OUT
BB1 (entry)	$\{e_1\}$ cioè $\{b-a\}$	$\{e_1\}$
BB2	$\{e_1\}$	$\{e_1\}$
BB3	$\{e_1, e_2\}$	$\{e_2\}$
BB4	$\{e_2\}$	$\emptyset$
BB5	$\{e_1\}$	$\emptyset$
BB6	$\emptyset$	$\{e_2\}$
BB7	$\{e_2\}$	$\emptyset$
BB8 (exit)	$\emptyset$	$\emptyset$

Tabella 3: Fixed point Very Busy Expressions

Interpretazione: all'entry l'unica espressione very busy è $b - a$. Si può fare **code hoisting** di $b - a$ prima del branch ($a \neq b$). **Convergenza:** 2 iterazioni.

3 Dominator Analysis

Analisi **forward** sul CFG di esempio. Un blocco X **domina** un blocco Y se ogni percorso da ENTRY a Y passa per X . Ogni blocco annota l'insieme $DOM[B]$ dei suoi dominatori.



$$DOM[F] = \{A, C, F\}$$

Figura 2: CFG Dominator Analysis

3.1 Struttura del CFG

Blocco	Predecessori	Successori
A (entry)	—	B, C
B	A	G
C	A	D, E
D	C	F
E	C	F
F	D, E	G
G (exit)	B, F	—

Insieme universale: $U = N = \{A, B, C, D, E, F, G\}$ (tutti i basic block).

3.2 Framework di Dataflow Analysis

Parametro	Valore
Dominio	$\mathcal{P}(\mathcal{B}), \mathcal{B} = \{A, \dots, G\}$
Direzione	Forward
Equazioni	$\text{OUT}(B) = f_B(\text{IN}(B)); \text{IN}(B) = \bigwedge_{P \in \text{pred}(B)} \text{OUT}(P)$
Transfer function	$f_B(x) = \text{Gen}_B \cup (x \setminus \text{Kill}_B) = \{B\} \cup \text{IN}(B)$
Meet operator ($\bigwedge$)	Intersezione $\cap$
Boundary condition	$\text{OUT}(\text{ENTRY}) = \text{OUT}(A) = \{A\}$
Initial interior points	$\text{OUT}(B) = \mathcal{U}$ per ogni $B \neq A$

Tabella 4: Formalizzazione Dominator Analysis

3.2.1 Gen e Kill

	Valore	Motivazione
Gen_B	$\{B\}$	ogni blocco domina sé stesso
Kill_B	$\emptyset$	l'esecuzione di B non invalida dominatori precedenti

La transfer function si semplifica in $\text{OUT}(B) = \text{IN}(B) \cup \{B\}$.

3.3 Algoritmo iterativo

Per ogni iterazione, si visitano i blocchi **dall'entry verso l'exit**:

1. $\text{IN}(B) = \bigcap_{P \in \text{pred}(B)} \text{OUT}(P)$ (se nessun predecessore: $\emptyset$)
2. $\text{OUT}(B) = \text{IN}(B) \cup \{B\}$

Inizializzazione: $\text{OUT}(A) = \{A\}; \text{OUT}(B) = \mathcal{U}$ per $B \in \{B, \dots, G\}$.

Nel calcolo delle iterazioni, $\text{IN}(B)$ usa i valori di OUT **all'inizio** dell'iterazione corrente (schema classico a **snapshot**), come richiesto dal template dell'assignment.

3.4 Tabella delle iterazioni

3.4.1 Iterazione 0 (inizializzazione)

Blocco	OUT
A	$\{A\}$
B ... G	N (universale)

3.4.2 Iterazione 1

Blocco	OUT (inizio)	IN (calcolato)	OUT (fine)	Ch.?
A	$\{A\}$	$\emptyset$	$\{A\}$	No
B	N	$\text{OUT}(A) = \{A\}$	$\{A, B\}$	Sì
C	N	$\text{OUT}(A) = \{A\}$	$\{A, C\}$	Sì
D	N	$\text{OUT}(C) = N$	N	No
E	N	$\text{OUT}(C) = N$	N	No
F	N	$\text{OUT}(D) \cap \text{OUT}(E) = N$	N	No
G	N	$\text{OUT}(B) \cap \text{OUT}(F) = N$	N	No

3.4.3 Iterazione 2

Blocco	OUT (inizio)	IN (calcolato)	OUT (fine)	Ch.?
A	$\{A\}$	$\emptyset$	$\{A\}$	No
B	$\{A, B\}$	$\{A\}$	$\{A, B\}$	No
C	$\{A, C\}$	$\{A\}$	$\{A, C\}$	No
D	N	$\text{OUT}(C) = \{A, C\}$	$\{A, C, D\}$	Sì
E	N	$\text{OUT}(C) = \{A, C\}$	$\{A, C, E\}$	Sì
F	N	$N \cap N = N$	N	No
G	N	$\text{OUT}(B) \cap \text{OUT}(F) = \{A, B\} \cap N = \{A\}$	$\{A, G\}$	Sì

3.4.4 Iterazione 3

Blocco	OUT (inizio)	IN (calcolato)	OUT (fine)	Ch.?
A ... E	(invariati)	—	—	No
F	N	$\text{OUT}(D) \cap \text{OUT}(E) = \{A, C, D\} \cap \{A, C, E\} = \{A, C\}$	$\{A, C, F\}$	Sì
G	$\{A, G\}$	$\text{OUT}(B) \cap \text{OUT}(F) = \{A, B\} \cap N = \{A\}$	$\{A, G\}$	No

3.4.5 Iterazione 4

Blocco	OUT (inizio)	IN (calcolato)	OUT (fine)	Ch.?
A ... F	(invariati)	—	—	No
G	$\{A, G\}$	$\text{OUT}(B) \cap \text{OUT}(F) = \{A, B\} \cap \{A, C, F\} = \{A\}$	$\{A, G\}$	No

Nessun blocco cambia — **fixed point raggiunto**.

3.5 Risultato finale

Blocco	IN	OUT (= DOM)
A (entry)	$\emptyset$	$\{A\}$
B	$\{A\}$	$\{A, B\}$
C	$\{A\}$	$\{A, C\}$
D	$\{A, C\}$	$\{A, C, D\}$
E	$\{A, C\}$	$\{A, C, E\}$
F	$\{A, C\}$	$\{A, C, F\}$
G (exit)	$\{A\}$	$\{A, G\}$

Tabella 9: Fixed point Dominator Analysis

Verifica esempio del prof: $\text{DOM}[F] = \{A, C, F\}$.

- **A** domina tutto (entry).
- **C** domina F: ogni percorso $A \rightarrow \dots \rightarrow F$ passa per C.
- **B** non domina F né G (percorso $A \rightarrow B \rightarrow G$ evita F).
- **D** e **E** non dominano F reciprocamente (rami alternativi).

Convergenza: 4 iterazioni.

4 Constant Propagation

Analisi **forward** sul CFG di esempio. Per ogni punto del programma si calcola l'insieme di coppie (**variabile, valore costante**) garantite su **tutti** i percorsi che raggiungono quel punto. L'analisi valuta anche espressioni binarie quando entrambi gli operandi hanno valore costante noto.

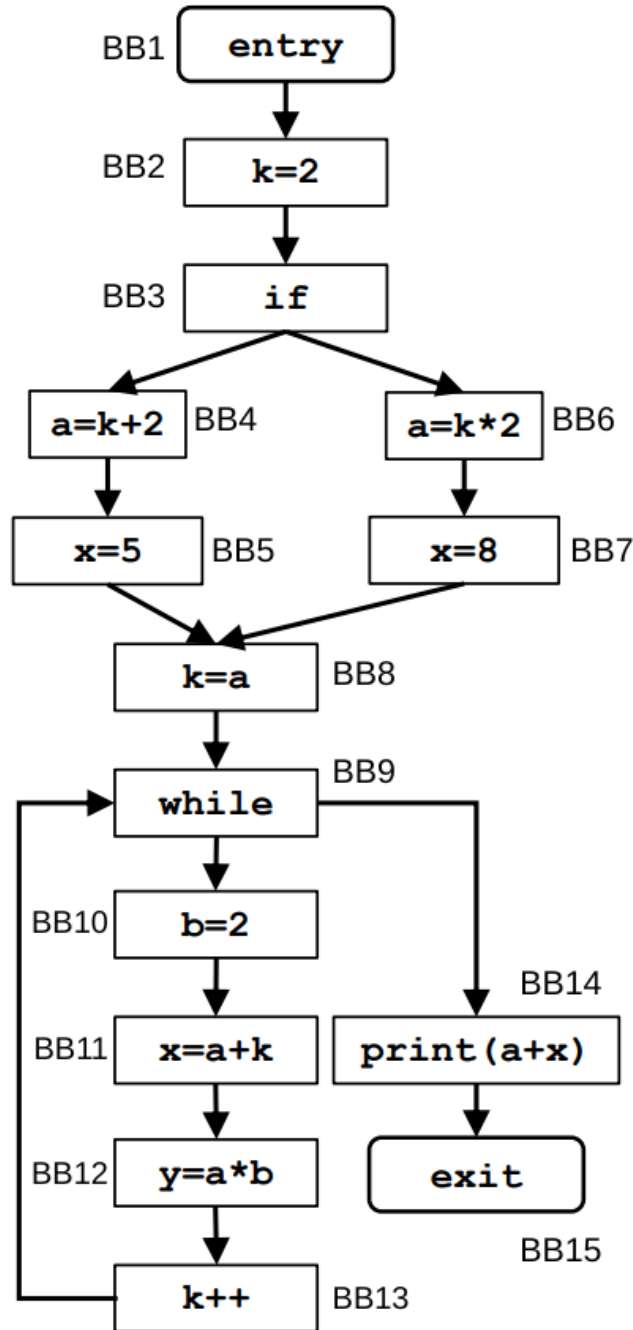


Figura 3: CFG Constant Propagation

4.1 Struttura del CFG

Blocco	Contenuto	Successori
BB1 (entry)	—	BB2
BB2	$k = 2$	BB3
BB3 (if)	condizione	BB4, BB6
BB4	$a = k + 2$	BB5
BB5	$x = 5$	BB8
BB6	$a = k * 2$	BB7
BB7	$x = 8$	BB8
BB8	$k = a$	BB9
BB9 (while)	condizione	BB10, BB14
BB10	$b = 2$	BB11
BB11	$x = a + k$	BB12
BB12	$y = a * b$	BB13
BB13	$k++$	BB9 (back-edge)
BB14	$\text{print}(a + x)$	BB15
BB15 (exit)	—	—

Variabili: $\{k, a, x, b, y\}$.

Notazione: $\text{OUT}(B)$ è un insieme di coppie, es. $\{(k, 2), (a, 4)\}$. Il simbolo $\mathcal{U}$ denota l'**universal set simbolico** (ogni variabile può valere qualsiasi costante).

4.2 Framework di Dataflow Analysis

Parametro	Valore
Dominio	$\mathcal{P}(\mathcal{V} \times \mathcal{C})$ — coppie (v, c)
Direzione	Forward
Equazioni	$\text{OUT}(B) = f_B(\text{IN}(B)); \text{IN}(B) = \bigwedge_{P \in \text{pred}(B)} \text{OUT}(P)$
Transfer function	$\text{OUT}(B) = \text{Gen}_B(\text{IN}) \cup (\text{IN}(B) \setminus \text{Kill}_B(\text{IN}))$ — dinamica
Meet operator ($\bigwedge$)	Intersezione $\cap$ con regole simboliche
Boundary condition	$\text{OUT}(\text{ENTRY}) = \text{OUT}(\text{BB1}) = \emptyset$
Initial interior points	$\text{OUT}(B) = \mathcal{U}$ per ogni $B \neq \text{BB1}$

Tabella 10: Formalizzazione Constant Propagation

4.2.1 Regole simboliche per $\cap$ e $\setminus$

Operazione	Risultato
$\mathcal{U} \cap S$	S
$\mathcal{U} \cap \mathcal{U}$	$\mathcal{U}$
$\{(v, c_1)\} \cap \{(v, c_2)\}$	$\{(v, c_1)\}$ se $c_1 = c_2$, altrimenti $\emptyset$
$S \setminus \{(v, C)\}$	rimuove ogni coppia con variabile v

Ridefinire v genera $\langle v, \mathcal{C} \rangle$ (tutte le costanti possibili per v). Esempio: $\{(k, 4)\} \setminus \{(k, C)\} = \emptyset$.

4.2.2 Transfer function dinamica

Per ogni istruzione nel blocco si aggiorna uno **stato locale** partendo da $\text{IN}(B)$:

Istruzione	Effetto
$v = c$ (costante)	kill v , gen (v, c)
$z = x \text{ op } y$	se (x, c_1) e (y, c_2) nello stato $\Rightarrow$ kill z , gen $(z, c_1 \text{ op } c_2)$
$z = x \text{ op } y$	operandi non costanti $\Rightarrow$ kill z , nessun gen
$k++$	kill k (incremento invalida la costante precedente)

4.3 Algoritmo iterativo

1. $IN(B) = \bigcap_{P \in \text{pred}(B)} OUT(P)$
2. Simula le istruzioni del blocco in ordine su $IN(B)$ (Gen/Kill dinamici)
3. $OUT(B) = \text{Gen} \cup (IN(B) \setminus \text{Kill})$

4.4 Tabella delle iterazioni

4.4.1 Iterazione 0 (inizializzazione)

Blocco	OUT
BB1	$\emptyset$
BB2 ... BB15	$\mathcal{U}$

4.4.2 Iterazione 1

BB	OUT (inizio)	IN (calcolato)	OUT (fine)	Ch.?
BB1	$\emptyset$	$\emptyset$	$\emptyset$	No
BB2	$\mathcal{U}$	$\emptyset$	$\{(k, 2)\}$	Sì
BB3	$\mathcal{U}$	$\{(k, 2)\}$	$\{(k, 2)\}$	Sì
BB4	$\mathcal{U}$	$\{(k, 2)\}$	$\{(k, 2), (a, 4)\}$	Sì
BB5	$\mathcal{U}$	$\{(k, 2), (a, 4)\}$	$\{(k, 2), (a, 4), (x, 5)\}$	Sì
BB6	$\mathcal{U}$	$\{(k, 2)\}$	$\{(k, 2), (a, 4)\}$	Sì
BB7	$\mathcal{U}$	$\{(k, 2), (a, 4)\}$	$\{(k, 2), (a, 4), (x, 8)\}$	Sì
BB8	$\mathcal{U}$	$OUT(BB5) \cap OUT(BB7) = \{(k, 2), (a, 4)\}$	$\{(k, 4), (a, 4)\}$	Sì
BB9	$\mathcal{U}$	$OUT(BB8) = \{(k, 4), (a, 4)\}$	$\{(k, 4), (a, 4)\}$	Sì
BB10	$\mathcal{U}$	$\{(k, 4), (a, 4)\}$	$\{(k, 4), (a, 4), (b, 2)\}$	Sì
BB11	$\mathcal{U}$	$\{(k, 4), (a, 4), (b, 2)\}$	$\{(k, 4), (a, 4), (b, 2), (x, 8)\}$	Sì
BB12	$\mathcal{U}$	$\{(k, 4), (a, 4), (b, 2), (x, 8)\}$	$\{(k, 4), (a, 4), (b, 2), (x, 8), (y, 8)\}$	Sì
BB13	$\mathcal{U}$	$\{(k, 4), (a, 4), (b, 2), (x, 8), (y, 8)\}$	$\{(k, 5), (a, 4), (b, 2), (x, 8), (y, 8)\}$	Sì
BB14	$\mathcal{U}$	$\{(k, 4), (a, 4)\}$	$\{(k, 4), (a, 4)\}$	Sì
BB15	$\mathcal{U}$	$\{(k, 4), (a, 4)\}$	$\{(k, 4), (a, 4)\}$	Sì

Dettaglio BB8: al merge, x vale 5 in BB5 e 8 in BB7 $\Rightarrow x$ non è costante in IN. Poi $k = a$ con $a = 4$ produce $OUT = \{(k, 4), (a, 4)\}$.

4.4.3 Iterazione 2

Il back-edge BB13 $\rightarrow$ BB9 fa convergere il loop: k non è più costante al join del while.

BB	OUT (inizio)	IN (calcolato)	OUT (fine)	Ch.?
BB1 ... BB8	(invariati)	—	—	No
BB9	$\{(k, 4), (a, 4)\}$	$OUT(BB8) \cap OUT(BB13) = \{(a, 4)\}$	$\{(a, 4)\}$	Sì
BB10	$\{(k, 4), (a, 4), (b, 2)\}$	$\{(a, 4)\}$	$\{(a, 4), (b, 2)\}$	Sì
BB11	$\{(k, 4), (a, 4), (b, 2), (x, 8)\}$	$\{(a, 4), (b, 2)\}$	$\{(a, 4), (b, 2)\}$	Sì
BB12	$\dots, (y, 8)\}$	$\{(a, 4), (b, 2)\}$	$\{(a, 4), (b, 2), (y, 8)\}$	Sì
BB13	$\{(k, 5), \dots, (y, 8)\}$	$\{(a, 4), (b, 2), (y, 8)\}$	$\{(a, 4), (b, 2), (y, 8)\}$	Sì
BB14	$\{(k, 4), (a, 4)\}$	$\{(a, 4)\}$	$\{(a, 4)\}$	Sì

BB15	$\{(k, 4), (a, 4)\}$	$\{(a, 4)\}$	$\{(a, 4)\}$	Sì
------	----------------------	--------------	--------------	----

Dettaglio BB9: $(k, 4)$ da BB8 e $(k, 5)$ da BB13 $\Rightarrow k$ non ha un unico valore $\Rightarrow$ nessuna coppia per k in IN. Resta $(a, 4)$.

4.4.4 Iterazione 3

Nessun blocco cambia — **fixed point raggiunto**.

4.5 Risultato finale

Blocco	IN	OUT
BB1	$\emptyset$	$\emptyset$
BB2	$\emptyset$	$\{(k, 2)\}$
BB3	$\{(k, 2)\}$	$\{(k, 2)\}$
BB4	$\{(k, 2)\}$	$\{(k, 2), (a, 4)\}$
BB5	$\{(k, 2), (a, 4)\}$	$\{(k, 2), (a, 4), (x, 5)\}$
BB6	$\{(k, 2)\}$	$\{(k, 2), (a, 4)\}$
BB7	$\{(k, 2), (a, 4)\}$	$\{(k, 2), (a, 4), (x, 8)\}$
BB8	$\{(k, 2), (a, 4)\}$	$\{(k, 4), (a, 4)\}$
BB9	$\{(a, 4)\}$	$\{(a, 4)\}$
BB10	$\{(a, 4)\}$	$\{(a, 4), (b, 2)\}$
BB11	$\{(a, 4), (b, 2)\}$	$\{(a, 4), (b, 2)\}$
BB12	$\{(a, 4), (b, 2)\}$	$\{(a, 4), (b, 2), (y, 8)\}$
BB13	$\{(a, 4), (b, 2), (y, 8)\}$	$\{(a, 4), (b, 2), (y, 8)\}$
BB14	$\{(a, 4)\}$	$\{(a, 4)\}$
BB15 (exit)	$\{(a, 4)\}$	$\{(a, 4)\}$

Tabella 13: Fixed point Constant Propagation

Interpretazione:

- $a = 4$ è garantito dopo il merge (BB8) e all'**exit** (**BB15**).
- k è costante (4) solo fino al loop; **k++** e il back-edge rendono k non costante in BB9.
- x non è costante nel loop (BB11 dipende da k non costante).
- $y = 8$ è costante nel corpo del loop ($a = 4, b = 2$).

Convergenza: 3 iterazioni.